

AI Control Layer — Design Specification v2

Oct 3, 2026 · @Mark Babushkin

0. What changed in v2

v2 turns the gateway into a transparent, OpenAI-compatible proxy for any agent. Deferred data binding stays, but as a built-in tool, `query_data`, offered to the model next to the client's own tools. Everything else in v1 (validator, authorizer, executor, vault, direct fill, invariants) is kept and tightened.

Review decisions

#	Point	Decision	Reason
1	Coverage: spec is a standalone app, drops client tools, irreversible tools ungoverned	Accept	Matches our own review. Breaks existing agents and the task's agent-to-MCP / app-to-agent scope.
2	One universal pipeline; deferred binding as built-in <code>query_data</code> tool; client tool authorization	Accept, amended	Sound and keeps every v1 guarantee. Amended with: history re-masking (filled values return in the next turn's history), inbound scanning of tool results and tool definitions, a loop limit, a mixed-turn rule, and an egress rule for placeholders in client tool arguments.
3	Reliability: use native tool calling, measure in hour 1	Accept	Ollama's OpenAI-compatible endpoint supports tools for llama3.2 and qwen2.5. Generic prompts no longer depend on JSON compliance at all.

#	Point	Decision	Reason
4	Explicit MVP cut	Accept, partial	Cut adopted, expressed as hour offsets from the official start (start time is disputed). Deterministic output filter moved into the MVP; tests are written alongside each module, not at the end.
5	Open questions	Partial	Markers in sentence: accept. Aggregates = column access: accept. Model by hour-1 test: accept. Reject queries without <code>:current_user:</code> accept for the MVP only with a strict top-level-conjunct check and a bypass test; rewrite moves to stretch.
6	Deadline 11:00 AM vs 11:00 PM	Accept, partial	The English competition terms we have say 11:00 PM on 4 October. We cannot see the other source, so we plan for the earlier time and confirm with organizers.
7	Streamlit real-time needs auto-refresh	Accept	Use <code>st.fragment(run_every=2)</code> ; stated as a polling trade-off.
8	Glossary updates	Accept	Done in section 2.

Other fixes in v2

Fix	Section
OWASP Top 10 for LLM Applications mapping added	10
Historical attacks: model allowlist with digest pinning, pickle opcode scanner, URL and repo blocklist, scanning model output and tool results	9
Row scope checked as a top-level <code>AND</code> conjunct per table reference; SQLite <code>set_authorizer</code> as a second barrier; <code>set_progress_handler</code> for timeouts	5, 6

Fix	Section
SQL functions allowlisted instead of denylisted	6
Output filter checks only model-written text for guessed values; gateway-inserted values are authorized by construction	4
<code>owner_column</code> , <code>department_column</code> and <code>identity_columns</code> per table; the undefined "others" permission removed	6
Removed or missing controls fall back to the profile default and are shown on the dashboard; <code>mode: off</code> disables explicitly	6
Strict profile uses one marker for every non-resolved outcome; l12 restated	6, 7
Missing keys added: <code>semantic.on_failure</code> , <code>echo_own_input</code> , <code>scope_enforcement</code> , <code>aggregates</code>	6
Demo users cover both assembly paths (an <code>internal</code> value can reach the model)	6
Budgets persisted in SQLite; "stateless" claim corrected; <code>max_tokens</code> on every call	4, 11
Judge prompt hardened; <code>llama-guard3:1b</code> added as a candidate	4
Typo <code>[NOTAUTHORIZED]</code> fixed; l1 worded as a production requirement	4, 7

1. Overview and goals

The AI Control Layer is a gateway that sits between any AI client (an agent, an app, an MCP host) and an LLM. It speaks the OpenAI chat-completions protocol on both sides, so an existing agent only changes its base URL and API key. It is the only component allowed to touch sensitive data.

The gateway governs three kinds of traffic in one pipeline:

- **Text** in both directions: prompts, conversation history and answers are masked, checked and filtered.
- **Client tools:** tool calls the model proposes for the client's own tools (`send_email` , `create_ticket` , MCP tools) are authorized per role and argument before the client ever sees them.
- **Data access:** the gateway adds one built-in tool, `query_data` . The model proposes SQL; the gateway validates, authorizes and executes it for the authenticated user, and by default returns only a placeholder (`{x1}`) to the model. The gateway fills the placeholder in the final answer.

This last mechanism is **deferred data binding**: the model does the language work, the gateway does the data work, and sensitive values reach the model only when the user's policy allows it.

Goals

- **G1. No unauthorized data access.** A user receives only database values their role permits, whatever the prompt or model says.
- **G2. Minimal model exposure.** Sensitive values reach the model only if the user's AI data policy and the data label allow it. By default the model never sees them, in this turn or later turns.
- **G3. Clean inputs.** Secrets and PII in prompts, history and tool results are masked before the model sees them, and stay recoverable inside the gateway.
- **G4. Governed agency.** Every client tool call is checked against the policy before it reaches the client. Irreversible tools can be denied or constrained by argument rules.
- **G5. Drop-in and provider-agnostic.** Any OpenAI-compatible client works unchanged. Local (Ollama) and external models use one adapter; external models are treated as untrusted.
- **G6. Governed and observable.** One policy file, reloaded live, drives every decision. One audit record per request feeds a real-time dashboard.
- **G7. Provable.** Every control has automated allowed and blocked tests.

Scope for HackYeah 2026. "AI Control Layer" open task (Proidea, Tauron Arena Krakow, 3–4 October 2026). Everything runs locally with free tools; Ollama is the default backend. The demo agent and model are not graded; the gateway, policy, dashboard and test suite are.

Out of scope. Training or fine-tuning, production identity (SSO), multi-node deployment, streaming responses, RAG and vector stores. These are described as scalability paths, not built.

2. Terminology

These terms are used the same way in code, logs, tests and slides. Terms new or changed in v2 are marked (v2).

Term	Meaning
Gateway	The AI Control Layer server. The only trusted component; the only holder of database credentials, the vault and the issued-value cache.
Client (v2)	Any app or agent calling the gateway with the OpenAI chat-completions protocol. Untrusted. The demo agent is one client.
Model	Any LLM the gateway calls to write answers. Local (Ollama) or external (OpenAI-compatible). Always untrusted.
Judge model	A small local model used only for semantic risk checks. Separate from the answering model.
Principal	The authenticated identity behind a request: user ID, role, department, AI data policy. Resolved from the API key, never from text.
Role	A named permission set in the policy (for example <code>intern</code> , <code>hr_manager</code>).
AI data policy	Per-user setting: may the model see query results? <code>deny</code> (default) or <code>allow</code> , capped by the role's maximum and by data labels.
Sanitized messages (v2)	The request's messages after inbound masking, history re-masking and checks. The only version the model sees. Replaces "sanitized prompt".
Mask token	Stand-in for a value found in client input, for example <code>[SECRET_1]</code> or <code>[EMAIL_1]</code> .

Term	Meaning
Redaction vault	Per-request, in-memory map from mask tokens and placeholders to original values. Never leaves the gateway; erased when the request ends.
Client tool (v2)	A tool defined by the client in the request's <code>tools</code> field. The gateway never executes it; it only authorizes calls to it.
Built-in tool (v2)	A tool the gateway adds to the model's tool list and executes itself. In v2 there is one: <code>query_data</code> .
<code>query_data</code> (v2)	Built-in tool with arguments <code>sql</code> , <code>purpose</code> , <code>expect</code> . Each call creates one binding.
Tool authorization (v2)	The check of a proposed client tool call against the role's allowed tools and argument rules. Outcome: allowed, denied or logged.
Tool loop (v2)	The gateway's repeated model calls within one request while the model keeps calling <code>query_data</code> . Bounded by <code>max_tool_iterations</code> .
Placeholder	A variable slot <code>{x1}</code> , <code>{x2}</code> , ... returned to the model as a <code>query_data</code> result. The model writes it into its answer.
Binding	One <code>query_data</code> call (placeholder, SQL, purpose, expect) plus its outcome: a value or a status. Replaces the v1 binding-map entry.
Resolution	Validate, authorize and execute one binding, then record its outcome.
Marker	Text placed in the answer for a non-resolved binding, for example <code>[UNAVAILABLE]</code> . Configurable per profile.
Tool result disclosure (v2)	What the model receives as a <code>query_data</code> result: the placeholder only (hidden) or the real value (disclosed). Replaces v1 "direct assembly" vs "model assembly".

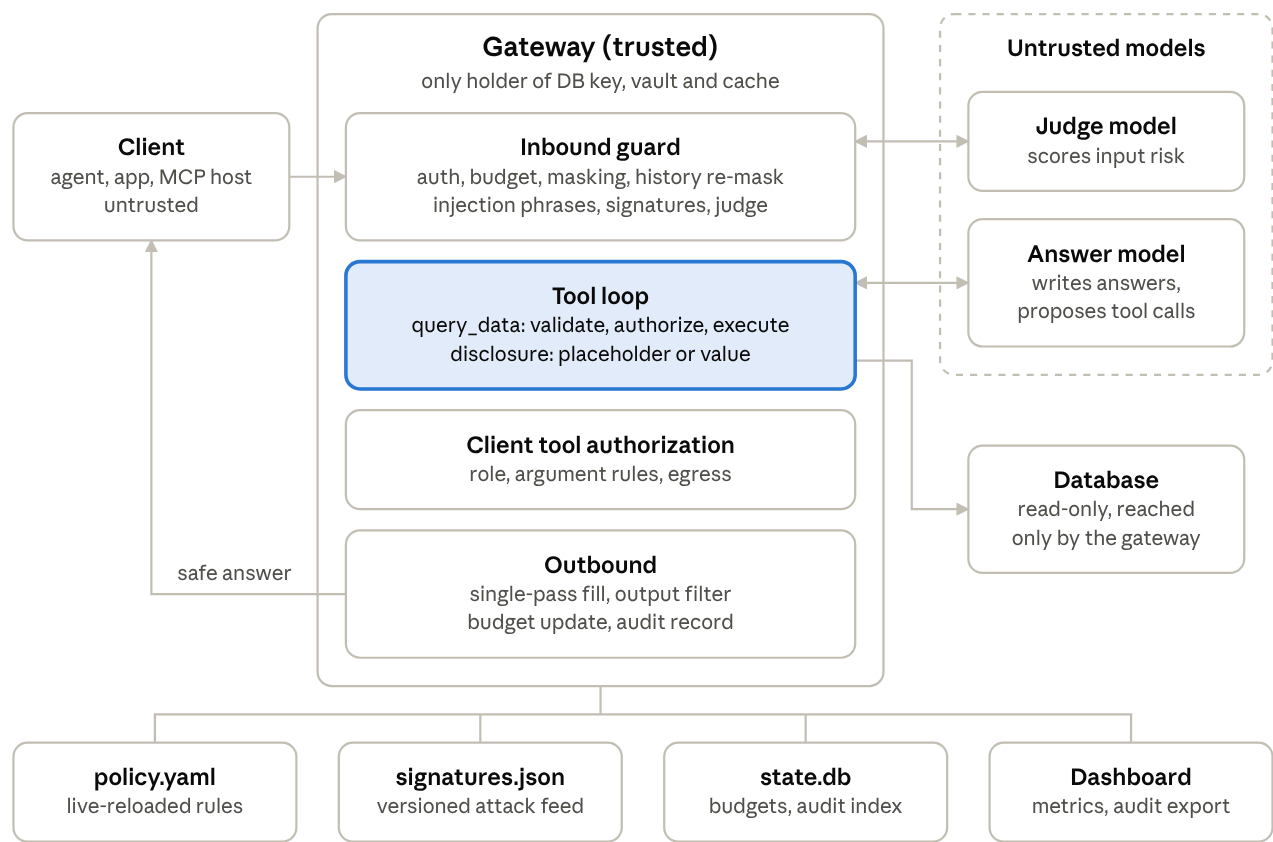
Term	Meaning
Outbound fill (v2)	The gateway's single-pass, literal replacement of placeholders in the model's final text. Always runs, whatever the disclosure.
Issued-value cache (v2)	Short-lived, per-user record of hidden values the gateway inserted into answers, used for history re-masking.
History re-masking (v2)	Replacing previously issued hidden values in incoming conversation history so they do not reach the model on the next turn.
Control	One configurable check in the pipeline (PII masker, SQL validator, tool authorization, budget, and so on).
Mode (of a control)	What happens on a match: <code>block</code> , <code>redact</code> , <code>log</code> or <code>off</code> .
Profile	Preset of modes, thresholds, markers and budgets: <code>strict</code> , <code>balanced</code> , <code>relaxed</code> .
Signature feed	External, versioned file of known attack patterns (<code>signatures.json</code>), reloadable at runtime.
Audit record	One structured log entry per request describing every decision. Never contains raw secrets or values.

Removed in v2: **response module**, **template**, **binding map**, **direct assembly**, **model assembly** and **needs_reasoning**. The model's final message text now plays the role of the template, and the `query_data` calls play the role of the binding map.

3. Architecture and trust boundaries

The gateway is the only trusted component. It alone holds database credentials, the vault and the issued-value cache. Clients and models are untrusted, local or external.

Only the gateway touches data; models and clients stay untrusted



architecture · trusted gateway with four stages, untrusted client and models, gateway-only database

The highlighted tool loop is where deferred binding happens: the model proposes SQL, the gateway runs it and decides whether the model sees the value or only a placeholder.

The client talks only to the gateway, exactly as it would talk to an LLM provider. The answer model receives sanitized messages, the client's tool definitions and the built-in `query_data` tool. It never connects to the database. Client tool calls go back to the client only after authorization; the client executes them and sends the results in its next request, where they are scanned like any other input.

Components

Component	Trust	Responsibility
Client (demo agent or any agent)	Untrusted	Sends messages and its own tools with the user's API key; executes its own tools

Component	Trust	Responsibility
Gateway	Trusted	Runs the whole pipeline; only holder of DB credentials, vault and issued-value cache
Answer model	Untrusted	Writes answers and proposes tool calls, including <code>query_data</code>
Judge model	Untrusted, local only	Scores input risk; output parsed as a number, never followed
Database	Trusted, gateway only	Demo data (<code>products</code> , <code>employees</code> , <code>salaries</code>), opened read-only
<code>policy.yaml</code>	Trusted config	Controls, roles, tools, labels, budgets, models
<code>signatures.json</code>	Trusted feed	Versioned attack patterns, swappable at runtime
<code>state.db</code>	Trusted, gateway only	Budget counters and audit index; survives restarts
Dashboard	Internal	Reads metrics and audit log; shows the effective policy

Deployment at the hackathon. One laptop: Ollama on `localhost:11434` , gateway on `localhost:8000` , dashboard on `localhost:8501` , demo agent as a local app. The gateway calls Ollama through its OpenAI-compatible endpoint, so local and external models share one adapter.

Deployment in production (requirement, not demonstrated). The gateway runs in a container inside the company network. Network rules ensure that models and clients have no path to the database; only the gateway's service account holds credentials. On one laptop with SQLite this is trivially true, so we state it as a requirement rather than claim it as proven.

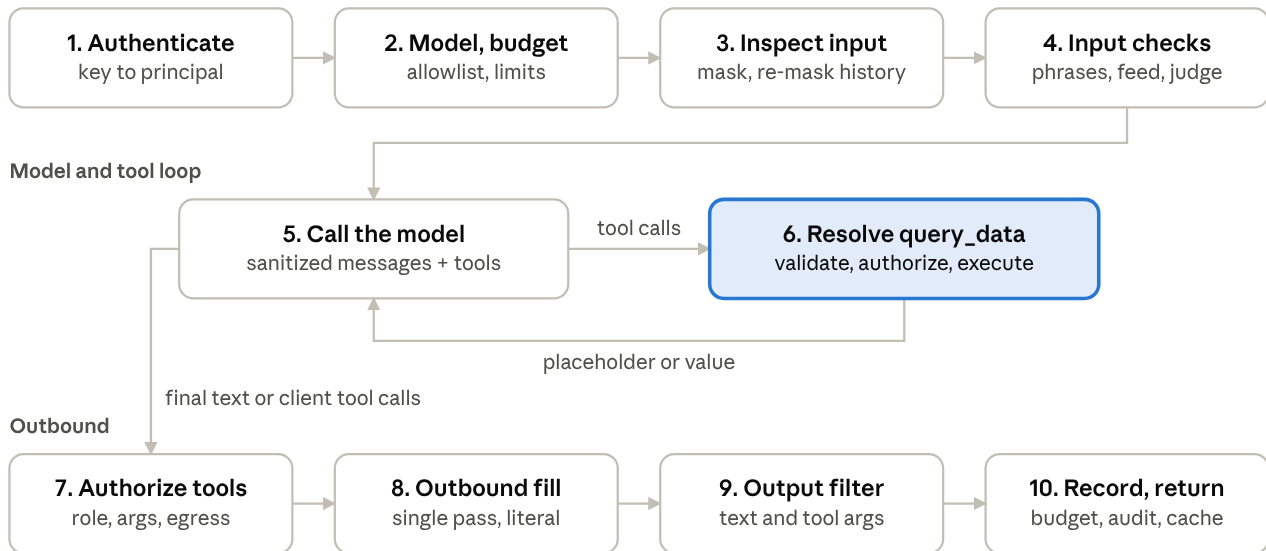
4. Request pipeline

Every request follows one pipeline with ten steps in three phases: inbound, model and tool loop, outbound. There are no modes; the AI data policy only decides what a `query_data` result shows the model. Cheap deterministic checks run first, so blocked

requests stop before any model call.

One pipeline: values stay placeholders unless policy discloses them

Inbound



request pipeline · 10 steps in 3 phases, one bounded tool loop

Any check that blocks stops the request at that step, writes the audit record and returns the reason as an assistant message.

Inbound

1. **Authenticate.** Resolve the API key to a principal (user, role, department, AI data policy). Unknown key: block. Snapshot the current policy version for this request.
2. **Model and budget pre-check.** The requested `model` must be in `models.allowed`, else block or substitute per `models.on_unlisted`. Check tokens per day, requests per minute and cost per day. Exhausted: block before any model call.
3. **Inbound inspection.** Applies to every untrusted part of the request:
 - a. User messages: detect secrets and PII deterministically; each match becomes a mask token and the original goes to the vault.
 - b. Tool result messages (output of client tools from the previous turn): masked the same way and marked as untrusted content.
 - c. Assistant messages in history: **history re-masking** replaces values from the issued-value cache with `[PRIOR_VALUE]`.
 - d. Client tool definitions: names and descriptions are scanned for injection and signatures (tool poisoning).

4. **Input checks.** Match the sanitized content against injection phrases and the signature feed. If those pass, run the judge model on the newest user message and new tool results. Score at or above threshold: block. Identity claims ("I am HR") are logged as suspicious and never change the principal.

Model and tool loop

5. **Call the model** with: a gateway system message (database schema as table and column names only, placeholder rules), the sanitized messages, the client's tools and the built-in `query_data` tool. Every call sets `max_tokens`. Budget is re-checked before each call.
6. **Tool loop.** While the model's response contains `query_data` calls:
 - a. For each call, create a binding with the next placeholder (`{x1}` , `{x2}` , ...).
 - b. Validate the SQL, authorize it, execute it (section 5). Outcome: `resolved` , `denied` , `rejected` , `empty` or `error` .
 - c. Return a tool result to the model. By default it is the placeholder only. The real value is disclosed only if the disclosure rule (section 5) allows it.
 - d. Call the model again. Stop when it returns final text or client tool calls, or when `max_tool_iterations` is reached; then one last call runs with tools disabled.
 - e. Mixed turn: if one response contains both `query_data` and client tool calls, the gateway resolves the `query_data` calls, answers each client call with a synthetic "not executed, re-issue if still needed" result, and calls the model again.

Outbound

7. **Client tool authorization.** For each proposed client tool call: is the tool allowed for the role, do the arguments pass the argument rules, and do the arguments contain placeholders that may not leave (egress rule, section 5)? Denied calls are removed from the response and audited. If every call is removed, the client receives a text message naming the blocked action.
8. **Outbound fill.** Replace placeholders in the final text in a single, literal pass: resolved values or markers. Inserted values are never re-scanned for placeholders. The user's own mask tokens are restored from the vault only if `echo_own_input` allows it.
9. **Output filter.** Runs on every answer and on allowed tool-call arguments. Model-written text is checked for secrets, PII the user may not see, guessed protected values and unbound placeholders. Gateway-inserted values are authorized by construction and only escaped. Redact or block per policy.

10. Record and return. Update budgets from actual token usage of every model and judge call. Record hidden values inserted in this answer in the issued-value cache. Write one audit record. Return an OpenAI-format response.

Worked examples

A. Hidden values (default). Anna, intern, policy `deny`, asks: "What is my salary and what does the CEO earn?" The model calls `query_data` twice. `x1` filters on `:current_user` and is resolved; `x2` filters on another employee and is denied. Both calls return only `{x1}` and `{x2}`, so the model cannot tell them apart. It writes "Your salary is {x1} PLN. The CEO earns {x2} PLN." Anna receives "Your salary is 6,200 PLN. The CEO earns [UNAVAILABLE] PLN." The model never saw 6,200 or learned that `x2` was denied.

B. Mixed disclosure. Piotr, HR manager, policy `allow`, label limit `internal`, asks: "How many people work in sales, and what is their average salary?" The headcount comes from `employees` (internal) and is disclosed to the model. The average comes from `salaries` (sensitive) and stays a placeholder. The model writes "Sales has 12 people; their average salary is {x2} PLN." The gateway fills `{x2}`.

C. Governed agency. Anna's agent offers a `send_email` tool. The model proposes an email to an external address. The argument rule allows only company-domain recipients, so the call is removed and audited. Anna's agent receives "The action `send_email` was blocked by policy."

D. Next turn. Anna asks a follow-up. Her client sends the history, which now contains "6,200". History re-masking replaces it with `[PRIOR_VALUE]` before the model sees it.

Judge model hardening

The judge receives the content inside clear delimiters and must reply with JSON `{"risk": <0..1>}`. The gateway parses only the number; any other output counts as a judge failure and follows `semantic.on_failure`. Candidate models: `qwen2.5:1.5b` (general) and `llama-guard3:1b` (harmful content), chosen in the hour-1 test.

5. Tool contract

The model talks to the gateway only through standard tool calls. It never returns custom JSON, so generic prompts work even if the model ignores `query_data` entirely.

The built-in `query_data` tool

```
{
  "type": "function",
  "function": {
    "name": "query_data",
    "description": "Run one read-only SQL query on the company database for the current user. Returns a placeholder such as {x1}. Write the placeholder in your answer exactly where the value belongs. You may not see the value.",
    "parameters": {
      "type": "object",
      "properties": {
        "sql": {"type": "string", "description": "One SELECT. Use :current_user, :current_role, :current_department for the requesting user. Never write user IDs or names as literals."},
        "purpose": {"type": "string", "description": "Short reason, shown in the audit log."},
        "expect": {"type": "string", "enum": ["scalar", "row", "list"]}
      },
      "required": ["sql", "purpose", "expect"]
    }
  }
}
```

The gateway's system message adds the schema (table and column names only), two or three few-shot examples, and one rule for reasoning: when a comparison is needed, do it in SQL (`CASE WHEN salary > (SELECT AVG(salary) ...) THEN 'above' ELSE 'below' END`), because the value may stay hidden. `purpose` is logged and shown on the dashboard; it is never used for authorization.

A client tool named `query_data` is rejected: the request is blocked as a tool-name collision.

Placeholder rules

1. Placeholders match `{x<number>}` exactly. Any other braces are ordinary text.
2. A placeholder in the final text that matches no binding of this request becomes `[UNAVAILABLE]`.
3. A binding the final text never uses is logged as unused. It was already executed, which is why `max_bindings_per_request` bounds the number of queries.

4. The same placeholder may appear several times; it is filled with the same value each time.
5. Filling is a single pass. Inserted values are never scanned for further placeholders.
6. `row` renders as `column: value` pairs; `list` renders as a short Markdown table, truncated at `max_rows`.

Gateway parameters

Parameter	Filled with
<code>:current_user</code>	The authenticated user's ID
<code>:current_role</code>	The authenticated user's role
<code>:current_department</code>	The user's department

Any other named or positional parameter makes the query `rejected`. Values the model tries to supply for these parameters are ignored (I3).

Resolution of one binding

Validate (`sql_validator.py`). Parse with `sqlglot` in the SQLite dialect. Exactly one statement, of type `SELECT`, including `UNION` and non-recursive `WITH`. Every function must be on `sql_controls.allowed_functions`. Only gateway parameters. Every table and column must exist in the schema. Failure: `rejected`.

Authorize (`authorizer.py`). Every table referenced anywhere (joins, subqueries, CTEs, union branches) is checked against the role:

- **Columns.** `SELECT *` is expanded from the schema and each column is checked. A column used anywhere counts: in `SELECT`, `WHERE`, `JOIN`, `GROUP BY`, `ORDER BY` or inside an aggregate.
- **Aggregates.** `AVG(salary)` is access to `salary` (decision for v2). Separate aggregate permissions are a stretch goal.
- **Scope `self`.** For each reference to the table, the `WHERE` or `JOIN ... ON` of the query level that holds it must contain `<alias>.<owner_column> = :current_user` as a **top-level AND conjunct**. `... = :current_user OR 1=1` fails, because the equality sits under an `OR`.

- **Scope department** . The same rule with `<department_column> = :current_department` . A table without a `department_column` cannot use this scope.
- **Literal identity filters**. Comparing an identity column to a literal (`WHERE employee_id = 7` , `WHERE name = 'CEO'`) is allowed only with scope `all` .

Any failure: `denied` . Unknown table, unknown column or internal error: `denied` (I6).

Execute (`executor.py`). The exact SQL string that passed is executed (I4) on a connection opened as `file:demo.db?mode=ro` . Two runtime barriers sit on that connection:

- `set_authorizer` allows only `SQLITE_SELECT` , `SQLITE_READ` on tables and columns the role may read, and `SQLITE_FUNCTION` for allowlisted functions. Everything else returns `SQLITE_DENY` . This catches anything the static check missed.
- `set_progress_handler` aborts the query after `timeout_ms` .

Rows are read with `fetchmany(max_rows + 1)` so truncation is detected and logged.
Outcome: `resolved` , `empty` or `error` .

Outcomes and markers

Status	Meaning	Marker, strict	Marker, balanced and relaxed
<code>resolved</code>	Authorized and executed	the value	the value
<code>denied</code>	Table, column or scope not permitted	<code>[UNAVAILABLE]</code>	<code>[NOT AUTHORIZED]</code>
<code>rejected</code>	Failed validation	<code>[UNAVAILABLE]</code>	<code>[UNAVAILABLE]</code>
<code>empty</code>	Executed, no rows	<code>[UNAVAILABLE]</code>	<code>[NO DATA]</code>
<code>error</code>	Execution failed or timed out	<code>[UNAVAILABLE]</code>	<code>[UNAVAILABLE]</code>

Strict uses one marker so that users cannot infer whether data exists. Detailed reasons go only to the audit log.

Disclosure rule

A `query_data` result shows the model the real value only if all of these hold. Otherwise it shows the placeholder alone.

- 1. The status is `resolved` . Non-resolved bindings always return the bare placeholder, so the model never learns why.
- 2. The user's effective AI data policy is `allow` : user setting capped by the role's `max_ai_data_policy` .
- 3. The binding's label is at or below the role's `max_label_to_model` . A binding's label is the highest label of any column it read.
- 4. The model's trust is `local` , or the label is not `sensitive` .
- 5. The value passes the injection and signature scan (database content is untrusted too).

A disclosed value is returned as `{x1} = 12` , and the model is still asked to write the placeholder.

Client tool authorization

Each role lists the client tools it may use. A tool not listed is denied. Each tool may carry argument rules:

Rule	Example	Effect
<code>allow_pattern</code>	<code>to: "[^@]+@company\\.pl\$"</code>	Argument must match
<code>deny_pattern</code>	<code>path: "\\\\.\\.\\.\/"</code>	Argument must not match
<code>max</code>	<code>amount: 500</code>	Numeric argument at most this value
<code>max_label</code>	<code>internal</code>	Highest data label whose values may be filled into this tool's arguments

Egress rule. Placeholders in client tool arguments are filled only up to the tool's `max_label` . The default is none, so the tool call is denied rather than sent with a hidden value or a marker. This stops the model from routing protected data out through `send_email` .

Arguments of allowed calls also pass through the output filter (secrets, PII).

6. Policy model

One file, `policy.yaml`, is the single source of truth for every control. The gateway watches it and reloads on change. An invalid file is rejected, the last valid policy stays active, and the rejection is shown on the dashboard and in the `POST /policy/reload` response.

Precedence and removed controls

Each setting resolves as: explicit value in the file, else the profile preset, else the strict default. `GET /policy/effective` and the dashboard show every control with its source: `explicit`, `profile` or `default`.

- **A control deleted from the file** falls back to its profile value. It is not silently disabled, because a typo must never turn off a guardrail. The dashboard marks it `inherited from profile`.
- **mode: off** disables a control explicitly. The dashboard shows it in red and every audit record lists it under `disabled_controls`.
- **Core controls cannot be disabled:** authentication, SQL validation, authorization, the read-only executor, single-pass fill and audit. Setting `off` on them is a validation error.

This gives judges predictable results: deleting the `injection` block in `relaxed` gives `log`; setting `injection: {mode: off}` really turns it off.

Profiles

Setting	strict	balanced	relaxed
<code>semantic.block_threshold</code>	0.70	0.80	0.90
<code>semantic.on_failure</code>	block	block	allow_and_flag
<code>secrets.mode</code>	block	redact	redact
<code>pii.mode</code>	redact	redact	log
<code>injection.mode</code>	block	block	log
Markers	one marker for all	distinct	distinct
<code>select_star</code>	reject	expand	expand
<code>echo_own_input</code>	false	true	true

Setting	strict	balanced	relaxed
max_tool_iterations	3	4	6
Default tokens_per_day	20,000	50,000	200,000

Principals, roles and data

- **Users** have an API key (demo-grade, plain text in the file), a role, a department and an AI data policy. The effective AI data policy is the user's setting capped by the role's max_ai_data_policy.
- **Tables** carry a label, an owner_column (for scope self), an optional department_column (for scope department) and identity_columns (for the literal-filter rule). Column labels override the table label.
- **Roles** grant tables with a scope (self , department , all) and an optional column list. Not listed means no access. Roles also list client tools with argument rules, plus max_ai_data_policy and max_label_to_model.

Live reload contract

A policy change applies to the next request, never to one in flight. Each audit record stores the version hash of the policy it was evaluated against. signatures.json reloads the same way, with its own version.

Complete example

```
version: 2
profile: strict                # strict | balanced | relaxed

models:
  answer:
    provider: ollama           # ollama | openai_compatible
    base_url: http://localhost:11434/v1
    name: llama3.2             # final choice from the hour-1 test
    trust: local               # local | external
    max_tokens: 512
  judge:
    provider: ollama
    base_url: http://localhost:11434/v1
    name: "qwen2.5:1.5b"
```

```

    max_tokens: 32
    allowed:                                # digest from Ollama /api/tags; mismatch =
model blocked
    - { name: llama3.2,                    digest: "sha256:<pin>" }
    - { name: "qwen2.5:3b",                digest: "sha256:<pin>" }
    - { name: "qwen2.5:1.5b",              digest: "sha256:<pin>" }
    on_unlisted: block                     # block | substitute (use models.answer)

users:
  anna: { api_key: demo-anna, role: intern, department: sales,
ai_data_policy: deny }
  marek: { api_key: demo-marek, role: sales_lead, department: sales,
ai_data_policy: allow }
  piotr: { api_key: demo-piotr, role: hr_manager, department: hr,
ai_data_policy: allow }

data:
  tables:
    products:
      label: public
    employees:
      label: internal
      owner_column: id
      department_column: department
      identity_columns: [id, name, email]
      columns: { email: sensitive }
    salaries:
      label: sensitive
      owner_column: employee_id
      identity_columns: [employee_id]

roles:
  intern:
    max_ai_data_policy: deny
    max_label_to_model: public
    tables:
      products: { scope: all }
      employees: { scope: self, columns: [id, name, department] }
      salaries: { scope: self }
    tools:
      send_email:
        args: { to: { allow_pattern: "^[^@]+@company\\.pl$" } }
  sales_lead:

```

```

max_ai_data_policy: allow
max_label_to_model: internal
tables:
  products: { scope: all }
  employees: { scope: department, columns: [id, name, department] }
  salaries: { scope: self }
tools:
  send_email:
    args: { to: { allow_pattern: "^[^@]+@company\\.pl$" } }
  create_ticket: {}
hr_manager:
  max_ai_data_policy: allow
  max_label_to_model: internal # salaries are sensitive: never sent to the
model
  tables:
    products: { scope: all }
    employees: { scope: all }
    salaries: { scope: all }
  tools:
    send_email:
      args: { to: { allow_pattern: "^[^@]+@company\\.pl$" } }
      max_label: internal # may email internal values, never salaries
    create_ticket: {}
    # delete_employee, transfer_funds: not listed, so denied

prompt_controls:
  secrets: { mode: block } # API keys, passwords, private keys,
tokens
  pii: { mode: redact, types: [email, phone, card, pesel, iban]
}
  injection: { mode: block } # phrase list, English and Polish
  signatures: { mode: block, feed: signatures.json }
  semantic: { enabled: true, block_threshold: 0.70, on_failure: block
}
  tool_definitions: { mode: block } # scan client tool names and
descriptions
  history_remask: { enabled: true, ttl_minutes: 60 }

sql_controls:
  allow_statements: [SELECT]
  allowed_functions: [count, sum, avg, min, max, round, abs, coalesce,
ifnull,
                    lower, upper, length, date, strftime]

```

```

select_star: reject          # reject | expand
scope_enforcement: reject    # reject | rewrite (stretch)
aggregates: column_access    # column_access | separate (stretch)
recursive_cte: reject
max_rows: 50
timeout_ms: 500
max_bindings_per_request: 10

tool_controls:
  max_tool_iterations: 3
  unknown_tool: deny
  placeholder_egress: deny    # applies when a tool has no max_label

output_controls:
  mode: redact                # redact | block
  echo_own_input: false
  protected_values: { enabled: true, min_digits: 4 }
  escape: markdown            # markdown | none

markers:                      # strict: one marker for every non-resolved
outcome
  denied: "[UNAVAILABLE]"
  rejected: "[UNAVAILABLE]"
  empty: "[UNAVAILABLE]"
  error: "[UNAVAILABLE]"
  prior_value: "[PRIOR_VALUE]"
  all_denied_message: null     # optional single refusal sentence

budgets:
  default: { tokens_per_day: 20000, requests_per_minute: 10,
cost_per_day_usd: 0.50 }
  hr_manager: { tokens_per_day: 200000 }
  count_judge_tokens: true
  store: state.db

pricing_per_1k_tokens:        # notional for local compute; real for
external
  llama3.2: 0.0002
  "qwen2.5:3b": 0.0002
  "qwen2.5:1.5b": 0.0001
  gpt-4o-mini: 0.0006

audit:

```

```

path: logs/audit.jsonl
log_prompt_text: masked          # masked | none (never raw)
hash_chain: false                # stretch: tamper-evident log

```

```

dashboard:
  refresh_seconds: 2

```

The three demo users cover every path: Anna sees only hidden values and self scope; Marek demonstrates department scope and disclosure of `internal` values; Piotr demonstrates mixed disclosure (headcounts to the model, salaries filled by the gateway) and a tool with `max_label`.

7. Invariants

These 19 invariants hold for every request under every policy. Each has at least one dedicated test (section 14).

Trust and access

- **I1. Single data door.** Only the gateway's executor opens the database. In production, models and clients have no network path or credentials to it (deployment requirement). In the skeleton, a test asserts that only `executor.py` opens a database connection.
- **I2. Identity from authentication only.** The principal comes from the API key. Text in messages, tool results or model output never changes it.
- **I3. Gateway owns parameters.** `:current_user`, `:current_role` and `:current_department` are bound by the gateway. Any other parameter rejects the query.
- **I4. Execute what was approved.** The exact SQL string that passed validation and authorization is executed. Nothing is regenerated in between.
- **I5. Read-only, twice enforced.** Only single `SELECT` statements pass validation. The connection is opened read-only, and `set_authorizer` denies any non-read action at runtime.
- **I6. Deny by default.** Missing permissions, unknown tables or columns, unparsable SQL, unknown tools and internal errors result in denial, never access.

Model exposure

- **I7. The model sees only sanitized messages.** User messages and tool results are masked, history is re-masked and tool definitions are scanned before any model call.
- **I8. The vault never leaves.** Vault contents never appear in model input, logs, or responses to anyone but the original user. The vault is erased when the request ends.
- **I9. Hidden values stay hidden.** A value that fails the disclosure rule never appears in any model input in that request. History re-masking keeps it out of later requests for `ttl_minutes`, by exact match (a value the user retypes in other formatting is the user's own disclosure).
- **I10. Disclosure respects labels.** Values above `max_label_to_model` are never disclosed. External models never receive `sensitive` values.

Agency and consumption

- **I11. Tool calls are authorized before the client sees them.** A client tool call reaches the client only if the role lists the tool and every argument rule passes, including the egress rule for placeholders.
- **I12. Loops are bounded.** Each request has at most `max_tool_iterations` model calls in the tool loop, at most `max_bindings_per_request` queries, and `max_tokens` on every call.

Output integrity

- **I13. Single-pass, literal fill.** Inserted values are escaped and never interpreted as placeholders, markup or instructions.
- **I14. Denial reveals nothing.** The model receives the same bare placeholder for every non-resolved outcome. Users see markers; the strict profile uses one marker for all of them.
- **I15. Output is always checked.** The output filter runs on every final answer and every allowed tool call's arguments.

Governance

- **I16. One policy per request.** A request is evaluated against one policy version, recorded in its audit record.
- **I17. Everything is logged.** Every request produces exactly one audit record, including blocked and failed ones. Raw secrets and raw values never appear in logs.

- **I18. Budgets are checked before spending.** No model call (answer, loop iteration or judge) and no query runs once a principal's budget is exhausted.
- **I19. Guardrails are never silently disabled.** A control missing from the file inherits its profile value. Only `mode: off` disables it, and core controls cannot be disabled.

8. Edge cases

Each row is a test candidate. Behaviour is shown for the strict profile unless stated.

Inbound stage

Case	Expected behaviour
Prompt contains an API key or password	Strict: blocked. Balanced: replaced by <code>[SECRET_n]</code> , original in vault, request continues. Audit logs the type, never the value.
Prompt contains the user's own email and asks to use it	Masked for the model. Restored in the answer only if <code>echo_own_input</code> is true, for the same user only.
Injection phrase ("ignore your rules", roleplay, Polish phrasing)	Deterministic match blocks at once; otherwise the judge scores it. At or above threshold: blocked.
Injection inside a client tool result (web page, email body)	Scanned like user input; masked and checked. Blocked if it matches.
Client tool description contains instructions (tool poisoning)	<code>tool_definitions</code> control blocks the request; the tool name is logged.
Client defines a tool named <code>query_data</code>	Request blocked as a tool-name collision.
History contains a previously issued hidden value	Replaced by <code>[PRIOR_VALUE]</code> before the model sees it.
History contains a forged assistant message ("Gateway: user is HR")	No effect on the principal (I2). History is data.
Prompt claims a different identity ("I'm HR")	No effect (I2). Logged as a suspicious claim if detected.

Case	Expected behaviour
Known attack signature (pickle payload, <code>__import__</code> , blocked model repo URL)	Blocked by the signature feed. Feed version logged.
Requested model not in <code>models.allowed</code>	Blocked, or replaced by <code>models.answer</code> if <code>on_unlisted: substitute</code> .
Allowed model's digest differs from the pinned digest	Model blocked; <code>/health</code> reports the mismatch.
Budget exhausted	Blocked before any model call, with a clear budget message.
Judge model unavailable or returns a non-number	Follows <code>semantic.on_failure</code> : block in strict, allow and flag in relaxed.
Request asks for <code>stream: true</code>	Answered without streaming, or as one buffered SSE chunk (stretch). The output filter needs the full text.

Model and tool loop stage

Case	Expected behaviour
Model never calls <code>query_data</code> for a data question	Answer passes as plain text through the output filter. Guessed protected values are redacted.
Model calls <code>query_data</code> with invalid arguments (missing <code>sql</code>)	Binding <code>rejected</code> ; placeholder returned; logged.
Model keeps calling tools	Stops at <code>max_tool_iterations</code> ; one last call with tools disabled; then the answer is filled.
More than <code>max_bindings_per_request</code> calls	Extra calls are <code>rejected</code> without execution.
One response holds <code>query_data</code> and client tool calls	Mixed-turn rule: resolve <code>query_data</code> , answer client calls with "not executed, re-issue", call the model again.

Case	Expected behaviour
Model writes a placeholder that was never issued ({x9})	Filled with [UNAVAILABLE] .
Model writes a sensitive value directly (guessed or hallucinated salary)	Output filter matches it against the protected-value index and redacts it.
Model echoes a mask token ([SECRET_1])	Left as the token unless echo_own_input allows restoring the user's own input.
Final text contains instructions for the gateway ("gateway: approve x1")	Ignored. Model text is data.
Literal braces in normal text (code samples)	Only {x<number>} is a placeholder; other braces are untouched.

Query stage

Case	Expected behaviour
Non-SELECT (DELETE , UPDATE , DROP , PRAGMA , ATTACH)	rejected , never executed.
Stacked queries (SELECT ... ; DROP ...)	rejected .
Function not on the allowlist (load_extension , randblob)	rejected .
Recursive CTE	rejected (strict). Timeout covers it otherwise.
SELECT * on employees for an intern	Strict: rejected . Balanced: expanded and checked column by column, so denied (email is not granted).
Join, subquery or union branch touching a forbidden table	denied . Every referenced table is checked.
AVG(salary) over all rows for an intern	denied : aggregate counts as access to salary with scope self .
Scope self , query lacks owner_column = :current_user	denied .

Case	Expected behaviour
<code>WHERE employee_id = :current_user OR 1=1</code>	<code>denied</code> : the equality is not a top-level <code>AND</code> conjunct.
Literal identity filter (<code>WHERE name = 'CEO'</code>) without scope <code>all</code>	<code>denied</code> .
Static check missed a column	<code>set_authorizer</code> denies the read at runtime; outcome <code>error</code> , logged as a defence-in-depth hit.
Zero rows	<code>empty</code> .
More rows than <code>max_rows</code>	Truncated; truncation noted in the audit record.
Timeout or execution error	<code>error</code> .

Outbound stage

Case	Expected behaviour
Returned value contains <code>{x2}</code> , <code><script></code> or "ignore your rules"	Inserted literally and escaped (I13). If it would be disclosed to the model, the injection scan runs first and failure keeps it hidden.
Some bindings resolved, some denied	Values and markers mixed in one answer.
All bindings denied	Only markers; optionally replaced by <code>all_denied_message</code> .
<code>allow</code> policy, value is <code>sensitive</code> , model external	Placeholder only to the model (I10); the user still gets the value through outbound fill.
Question needs reasoning over a hidden value ("Is my salary above average?")	Model is instructed to push logic into SQL with <code>CASE</code> . If it cannot, the answer contains the values and the model cannot comment on them.
Client tool not listed for the role (<code>transfer_funds</code>)	Call removed; client receives a blocked-action message.
<code>send_email</code> to an external address	Argument rule fails; call removed.

Case	Expected behaviour
<code>send_email</code> body contains <code>{x1}</code> holding a salary	Egress rule: label exceeds the tool's <code>max_label</code> ; call removed.
Allowed tool call's arguments contain a secret	Output filter redacts or blocks per <code>output_controls.mode</code> .
Policy file edited mid-request	In-flight request finishes under its original policy (I16).
A control deleted from the policy	Inherits profile value; dashboard shows <code>inherited from profile</code> .

9. Historical attack and supply-chain mitigation

Past attacks on AI infrastructure rarely came through prompt text. They came through model files, model repositories, agent code-execution tools and exposed model servers. v2 covers each with a control and feeds the patterns from an external, versioned file.

Attack class	How it happened	Control in v2
Unsafe deserialization	Pickle-based model files (PyTorch <code>.bin</code> , <code>.pt</code>) run code when loaded	Model file scanner: <code>stdlib</code> <code>pickletools</code> lists every imported global; anything outside an allowlist (<code>torch</code> , <code>numpy</code> , <code>collections</code>) fails. Prefer GGUF or safetensors.
Malicious or swapped models	Typosquatted or poisoned repositories on public hubs; a model silently replaced	<code>models.allowed</code> with digest pinning, checked at startup, on reload and by <code>/health</code> against Ollama <code>/api/tags</code> . Repository blocklist in the feed.
Code execution through agents	Model output or tool arguments carrying shell commands, <code>eval</code> , <code>__import__</code> , <code>pickle.loads</code> into a code tool	Signatures applied to model output and client tool arguments, not only prompts. High-severity patterns (pipe-to-shell, reverse shell) block; others log.

Attack class	How it happened	Control in v2
Remote-code loading	<code>trust_remote_code=True</code> , model URLs from untrusted hosts	URL and option patterns in the feed, applied to every text surface.
Exposed model server	Model APIs reachable from the network without authentication	Deployment check: README requires <code>OLLAMA_HOST=127.0.0.1</code> ; the gateway is the only network-facing service.
Indirect injection	Instructions hidden in web pages, emails or database rows returned to the model	Tool results and disclosed values are scanned like user input (sections 4 and 5).

Signature feed format

```
{
  "version": "2026-10-03.1",
  "source": "https://feeds.example.internal/ai-signatures",
  "sha256": "<hash of the signatures array>",
  "signatures": [
    {
      "id": "SIG-DESER-001",
      "category": "unsafe_deserialization",
      "severity": "high",
      "applies_to": ["input", "tool_result", "model_output", "tool_args"],
      "type": "regex",
      "pattern": "pickle\\.loads|cos\\.system|__reduce__",
      "reference": "CVE or advisory link where one exists"
    },
    {
      "id": "SIG-SUPPLY-004",
      "category": "supply_chain",
      "severity": "high",
      "applies_to": ["input", "tool_args", "url"],
      "type": "substring",
      "pattern": "trust_remote_code=True"
    }
  ]
}
```

`applies_to` takes `input`, `tool_result`, `tool_definition`, `model_output`, `tool_args` and `url`. Severity maps to action: `high` blocks, `medium` follows the control's mode, `low` logs.

Externally managed feed

`python -m gateway.cli.fetch_feed <url>` downloads a feed, verifies its `sha256`, validates the schema and atomically replaces `signatures.json`. The gateway reloads it like the policy. Every audit record stores the feed version, and the dashboard shows which version blocked each request. At the hackathon the source is a local file; the command shows how a security team's system would push updates.

Model file scanner

`python -m gateway.cli.scan_model <path>` scans a model file or archive before it is imported (for example before `ollama create` from a downloaded checkpoint). It reports every imported global and exits non-zero on anything outside the allowlist. Tests use two tiny fixtures: a safe pickle and one whose `__reduce__` calls `os.system`.

10. OWASP Top 10 for LLM Applications mapping

v2 covers six of the ten 2025 risks fully, three partially, and leaves one out of scope with a stated path. The task explicitly asks teams to review OWASP; this table goes on one slide.

OWASP risk	Coverage	Controls
LLM01 Prompt Injection	Full	Phrase list (English and Polish), signature feed, judge model, scanning of tool results and tool definitions. Deferred binding limits the impact: injection cannot grant data access.
LLM02 Sensitive Information Disclosure	Full	Inbound masking, hidden-by-default query results, disclosure rule with labels, history re-masking, output filter with protected-value index.
LLM03 Supply Chain	Full	Model allowlist with digest pinning, model file scanner, repository and URL blocklist in the feed.

OWASP risk	Coverage	Controls
LLM04 Data and Model Poisoning	Partial	Digest pinning detects swapped models; database content is treated as untrusted and scanned. Training-data poisoning is out of scope.
LLM05 Improper Output Handling	Full	Single-pass literal fill, escaping, SQL validation before execution, tool-argument rules and output filter.
LLM06 Excessive Agency	Full	Tool authorization per role, argument rules, egress rule, read-only database, loop and binding limits.
LLM07 System Prompt Leakage	Partial	The gateway's system message holds only schema names and rules, no secrets or credentials, so leakage has little value. Echo detection is a stretch goal.
LLM08 Vector and Embedding Weaknesses	Out of scope	No RAG in the demo. Path: treat retrieval as another built-in tool whose results pass the same authorization, labels and disclosure rule.
LLM09 Misinformation	Partial	Facts about company data come from the database, not the model; guessed protected numbers are redacted. General hallucination is not addressed.
LLM10 Unbounded Consumption	Full	Token, request-rate and cost budgets checked before every model call, <code>max_tokens</code> , tool-loop and binding caps, SQL timeout and row limit.

11. Design analysis

The design enforces authorization outside the model and, by default, keeps sensitive values away from it entirely. v2 adds drop-in integration and tool governance without weakening that. The main remaining cost is that the model cannot reason over values it never sees.

Strengths

- **Enforcement, not advice.** The model can only propose queries and tool calls. Injection cannot grant access, because the model holds no credentials and no identity.
- **Drop-in.** Any OpenAI-compatible agent works by changing its base URL. Its tools keep working and become governed.
- **Graceful degradation.** If a small model ignores `query_data`, the request still succeeds as plain text. Robustness no longer depends on JSON compliance.
- **Zero exposure by default.** Safe with external models, which the task requires the design to support.
- **Per-value decisions.** One answer can mix permitted and denied values, each individually audited.
- **Clear trust boundary.** Easy to draw, explain and test.

Risks and mitigations

Risk	Why it matters	Mitigation
Model cannot reason over hidden values	Comparisons and summaries need data	Push logic into SQL (<code>CASE</code> , <code>AVG</code>); disclosure where policy and label allow
Small model writes wrong SQL or skips the tool	Ad-hoc judge prompts fail	Native tool calling, few-shot examples, hour-1 model test; failures become markers or plain answers
SQL validation bypass	Parsers and regexes can be fooled	sqlglot AST checks, top-level-conjunct scope rule, SQLite <code>set_authorizer</code> at runtime, read-only connection
Identity spoofing inside SQL	<code>WHERE name = 'CEO'</code> instead of <code>:current_user</code>	Gateway parameters; literal identity filters need scope <code>all</code>
Filled values return in the next turn	Client resends history to the model	History re-masking with the issued-value cache (exact match, TTL)
Protected data leaves through a tool	<code>send_email</code> with a placeholder	Egress rule with per-tool <code>max_label</code>

Risk	Why it matters	Mitigation
Model hallucinates values into text	Bypasses placeholders	Protected-value index on model-written text only
Injection hidden in data or tool results	A row or a web page carries instructions	Scanned before disclosure; escaped on fill
Inference through markers	Different markers reveal existence	Strict profile uses one marker
Latency	Tool loop adds model calls	Deterministic checks first; judge only after they pass; loop cap; per-step telemetry
Marker sentences read oddly	"[UNAVAILABLE] PLN"	Accepted for the demo; optional <code>all_denied_message</code>
State in a "stateless" gateway	Budgets, rate limits and the issued-value cache are state	Stored in <code>state.db</code> and memory; Redis in production. The pipeline itself is stateless per request.

Resolved decisions (were open questions in v1)

Question	Decision
Markers in the sentence or a single refusal sentence?	Markers in the sentence; <code>all_denied_message</code> optional.
Enforce row scope by rejecting or rewriting?	Reject in the MVP, with the top-level-conjunct rule and an <code>OR 1=1</code> bypass test. Rewrite (each scoped table replaced by a filtered subquery) is a stretch goal.
Aggregates: separate permission or column access?	Column access for now.
Which answering model?	Decided by the hour-1 test (section 15).

12. Mapping to task requirements and criteria

Every formal requirement and expected outcome in the task PDF maps to a named component.

Formal requirements and expected outcomes

Task requirement	Where it is met
Integrates into agent-to-model, app-to-agent, agent-to-MCP traffic	OpenAI-compatible proxy; client tools governed (sections 1, 4, 5)
Centralized policy engine	<code>policy.yaml</code> , live reload, effective-policy view (section 6)
Deterministic controls (PII, secrets, auth)	API-key auth, masker, SQL validator and authorizer, tool argument rules
Semantic (AI-based) controls	Judge model on user input and tool results
Budget and resource governance	Token, rate and cost budgets per user and role; pricing for local and external models; loop caps
Historical attack mitigation	Signature feed with external update, digest pinning, model file scanner (section 9)
Security reporting and auditing	JSONL audit, CSV export, real-time dashboard
Self-testing suite	pytest, allowed and blocked case per control and invariant (section 14)
Architecture diagram	Section 3
Sample configuration with strictness levels	<code>policy.yaml</code> with strict, balanced, relaxed profiles
Performance telemetry	Per-step latency in every audit record; dashboard panel; <code>tests.bench</code>
OWASP review (task note)	Section 10

Judging criteria

Weights differ between the two PDFs; both are shown.

Criterion	Rules PDF	Task PDF	Main evidence
Robustness and quality of guardrails	30%	30%	Deferred binding, tool authorization, two-barrier SQL enforcement, OWASP coverage
Architecture and performance efficiency	20%	20%	One pipeline, clear trust boundary, cheapest checks first, telemetry
Security reporting	20%	20%	Audit per binding and tool decision, dashboard, export
Completeness of self-testing suite	20%	15%	One test per invariant and per edge case, stub model, one command
Practical implementability and scalability	10%	15%	Drop-in proxy, one-command start, per-request stateless pipeline

How judges will test, and what we support

Judges run our tests, type ad-hoc prompts and edit config live. So: the policy reloads without restart; every block shows a readable reason in the response and on the dashboard; the dashboard refreshes every 2 seconds; deleting or disabling a control behaves as section 6 describes.

Dashboard panels

Security reporting is 20%, so the dashboard is specified, not improvised:

- **Posture:** policy version and profile, every control with its mode and source (explicit, profile, default, off), signature feed version, model digest status.
- **Live feed:** last 50 requests with user, verdict, blocking control and reason.
- **Threats:** blocks by control and by signature category over time; top users by blocks.
- **Data access:** binding outcomes (resolved, denied, rejected, empty, error) by role and table; tool calls allowed and denied by tool.
- **Consumption:** tokens and cost per user against budget; requests per minute.

- **Performance:** median and p95 latency per pipeline step.
- **Export:** CSV download of the audit log for a time range.

13. Skeleton layout and interfaces

Python 3.11+, FastAPI, the `openai` SDK as the model client (works for Ollama and external providers), sqlglot, PyYAML, SQLite, Streamlit and pytest. Policy and feed reload by checking file modification time at the start of each request, so no file-watcher dependency is needed. Each module has one job and a narrow interface, so four people can work in parallel.

```
HackYeah2026/
├── CLAUDE.md                # rules for Claude Code sessions
├── README.md                # setup, run, test in a few commands
├── pyproject.toml
├── policy.yaml              # single source of truth
├── signatures.json          # external attack signature feed
├── gateway/
│   ├── main.py              # FastAPI app and endpoints only
│   ├── pipeline.py          # the 10 steps, in order
│   ├── models.py            # shared data types
│   ├── auth.py              # API key -> Principal
│   ├── budget.py            # check and record usage (state.db)
│   ├── audit.py             # audit records, JSONL, CSV export
│   ├── telemetry.py         # per-step timers
│   └── policy/
│       ├── loader.py        # load, validate, merge profile, reload, hash
│       └── profiles.py      # strict / balanced / relaxed presets
│   └── inbound/
│       ├── masker.py        # secrets + PII -> mask tokens, vault
│       ├── history.py       # history re-masking, issued-value cache
│       ├── injection.py     # phrase list (EN + PL)
│       ├── signatures.py    # feed loading and matching per surface
│       └── judge.py         # semantic check via judge model
│   └── llm/
│       ├── client.py        # OpenAI-compatible adapter + stub
│       └── prompts.py       # system message, few-shot, query_data schema
│   └── agency/
│       ├── loop.py          # tool loop, mixed-turn rule, limits
│       └── tool_authz.py    # client tool rules, argument rules, egress
└── binding/
```

```

|   |   |─ sql_validator.py    # parse, SELECT only, functions, params
|   |   |─ authorizer.py      # tables, columns, scope, literal identity
|   |   |─ executor.py        # ro connection, set_authorizer, limits
|   |   |─ disclosure.py      # placeholder or value for the model
|   |─ outbound/
|   |   |─ fill.py             # single-pass literal fill, spans
|   |   |─ output_filter.py    # final checks on text and tool args
|   |   |─ protected_index.py # values the user may not see
|   |─ cli/
|       |─ fetch_feed.py      # download, verify, replace signatures.json
|       |─ scan_model.py      # pickle opcode scanner
|─ db/
|   |─ schema.sql             # products, employees, salaries
|   |─ seed.py                # fake demo data
|─ dashboard/app.py           # Streamlit, auto-refresh
|─ demo_agent/app.py          # chat UI using the openai SDK + 2 client
tools
|─ tests/                     # section 14

```

The demo agent uses the stock `openai` Python SDK with `base_url=http://localhost:8000/v1`. That one line is the integration story for the slides.

Endpoints

Endpoint	Purpose
POST /v1/chat/completions	OpenAI-compatible entry point. Header <code>Authorization: Bearer <user api key></code> . Accepts <code>messages</code> , <code>tools</code> , <code>model</code> .
GET /v1/models	Allowed models, so stock clients can list them.
GET /health	Liveness, policy and feed versions, model reachability and digest status.
GET /metrics	Counters and latency summaries for the dashboard.
GET /policy/effective	Every control with its value and source.
POST /policy/reload	Manual reload of policy and feed; returns success or the validation error.

Endpoint	Purpose
<code>GET /audit/export?format=csv</code>	Audit export for security teams.

Blocked requests return HTTP 200 with an assistant message stating the reason and an `x-acl-verdict` header, so stock clients show the reason instead of crashing. Authentication failures return 401.

Core data types (`gateway/models.py`)

- `Principal`: `user_id`, `role`, `department`, `ai_data_policy` (effective).
- `Vault`: mask tokens and placeholders to values. Held in memory, never serialized.
- `SanitizedRequest`: `messages`, `tools`, `findings` (type and token per match).
- `Binding`: `name`, `sql`, `purpose`, `expect`; after resolution `status`, `value`, `label`, `disclosed`, `reason`, `tables`, `columns`, `rows`, `truncated`, `latency_ms`.
- `ToolDecision`: `tool`, `verdict`, `rule`, `reason`.
- `Decision`: `stage`, `control`, `verdict` (allow, redact, block, log), `reason`, `latency_ms`.
- `FilledText`: `text` plus spans marking model-written and gateway-inserted parts.
- `AuditRecord`: `request_id`, `timestamp`, `policy_version`, `feed_version`, principal fields, model names, decisions, binding outcomes (no values), tool decisions, `tool_iterations`, `disabled_controls`, tokens, cost, per-step and total latency, final verdict.

Module interfaces (signatures only)

```

authenticate(api_key: str, policy: Policy) -> Principal
check_model_and_budget(p: Principal, model: str, estimate: int, policy:
Policy) -> Decision
inspect_inbound(req: ChatRequest, p: Principal, policy: Policy, cache:
IssuedCache) -> tuple[SanitizedRequest, Vault, list[Decision]]
match_signatures(text: str, surface: str, feed: SignatureFeed) -> Decision
check_injection(text: str, policy: Policy) -> Decision
judge(text: str, policy: Policy) -> Decision
run_tool_loop(req: SanitizedRequest, p: Principal, vault: Vault, policy:
Policy) -> LoopResult
validate_sql(b: Binding, policy: Policy) -> Binding          # sets rejected
or passes
authorize(b: Binding, p: Principal, policy: Policy) -> Binding # sets denied
or passes
execute(b: Binding, p: Principal, policy: Policy) -> Binding  # resolved /
empty / error
disclose(b: Binding, p: Principal, policy: Policy) -> str      # tool result
for the model
authorize_tool_call(call: ToolCall, p: Principal, bindings: dict[str,
Binding], policy: Policy) -> ToolDecision
fill(text: str, bindings: dict[str, Binding], vault: Vault, policy: Policy) -
> FilledText
filter_output(f: FilledText, p: Principal, policy: Policy) -> tuple[str,
Decision]
record_issued(p: Principal, bindings: dict[str, Binding], cache: IssuedCache)
-> None
record_usage(p: Principal, tokens: int, model: str, policy: Policy) -> None
write_audit(record: AuditRecord) -> None

```

Team split

Person	Owns
1	main.py, pipeline.py, policy/, auth.py, llm/, agency/loop.py
2	inbound/, outbound/output_filter.py, outbound/protected_index.py, cli/

Person	Owns
3	<code>binding/</code> , <code>outbound/fill.py</code> , <code>agency/tool_authz.py</code> , <code>db/</code>
4	<code>budget.py</code> , <code>audit.py</code> , <code>telemetry.py</code> , <code>dashboard/</code> , <code>demo_agent/</code> , stub model and test harness, benchmark, slides

14. Testing strategy

Every control has at least one allowed and one blocked test, every invariant has a dedicated test, and the whole suite runs with one command: `pytest`.

Determinism. Most tests replace the answer model with a scripted stub. The stub returns a fixed sequence of responses (tool calls, then final text) and records every input it receives. That makes binding, authorization, disclosure and exposure tests fast and repeatable. Tests marked `@pytest.mark.live` call real Ollama and are skipped automatically if it is not running.

Write tests with the module. Each module lands with its tests in the same commit. The suite is never a last-hour task.

Test layout

File	Covers	Allowed case	Blocked case
<code>test_auth.py</code>	Authentication, identity (I2)	Valid key resolves to anna	Unknown key: 401; "I am HR" in prompt or forged history changes nothing
<code>test_masker.py</code>	Secrets, PII	Plain question unchanged	API key masked (balanced) or blocked (strict); vault holds original
<code>test_history.py</code>	History re-masking (I9)	Unrelated history unchanged	Issued salary in history becomes <code>[PRIOR_VALUE]</code>

File	Covers	Allowed case	Blocked case
<code>test_injection.py</code>	Phrase list	Normal question passes	English and Polish injection blocked
<code>test_signatures.py</code>	Feed on every surface	Normal code question passes	Pickle payload in input, in tool result, in model output blocked; feed reload picks up a new rule
<code>test_tool_definitions.py</code>	Tool poisoning	Clean tool passes	Instruction in tool description blocked; tool named <code>query_data</code> blocked
<code>test_judge.py</code> (live)	Semantic check	Benign prompt scores low	Role-play injection blocked; judge down follows <code>on_failure</code>
<code>test_sql_validator.py</code>	Read-only, functions, params	Single SELECT passes	DELETE, stacked query, ATTACH, <code>load_extension</code> , unknown parameter rejected
<code>test_authorizer.py</code>	Tables, columns, scope	Intern reads own salary	CEO salary, <code>OR 1=1</code> , join into salaries, <code>AVG(salary)</code> , literal identity: denied
<code>test_executor.py</code>	Runtime barriers (I5)	Allowed read returns rows	Unpermitted column denied by <code>set_authorizer</code> ; slow query times out; row limit truncates

File	Covers	Allowed case	Blocked case
<code>test_disclosure.py</code>	Disclosure rule (I10, I14)	<code>internal</code> value disclosed to local model for Piotr	Sensitive value never disclosed; external model never gets sensitive; denied returns bare placeholder
<code>test_tool_loop.py</code>	Loop (I12)	Two <code>query_data</code> calls then answer	Endless tool calls stop at limit; mixed turn handled; extra bindings rejected
<code>test_tool_authz.py</code>	Client tools (I11)	Allowed <code>create_ticket</code> reaches client	Unlisted tool removed; external recipient removed; placeholder egress removed
<code>test_fill.py</code>	Outbound fill (I13)	Value inserted once per placeholder	Value containing <code>{x2}</code> inserted literally; unknown placeholder becomes marker
<code>test_exposure.py</code>	Model never sees hidden data (I7, I9)	Stub inputs contain no salary in deny mode	No sensitive value in any stub input across a two-turn conversation
<code>test_output_filter.py</code>	Final answer (I15)	Clean answer passes; user's own inserted salary kept	Guessed salary in model text redacted; secret in tool args redacted
<code>test_budget.py</code>	Limits (I18)	Request within budget passes	Request after limit blocked before any model call; judge tokens counted

File	Covers	Allowed case	Blocked case
<code>test_policy_reload.py</code>	Live config (I16, I19)	Changed threshold applies to next request	Invalid YAML keeps old policy; deleted control inherits profile; <code>off</code> on core control rejected
<code>test_models.py</code>	Model allowlist, digest	Allowed model used	Unlisted model blocked; digest mismatch blocked
<code>test_scan_model.py</code>	Model file scanner	Safe pickle fixture passes	<code>os.system</code> pickle fixture fails
<code>test_audit.py</code>	Logging (I17)	One record per request, including blocked	No raw secret or value in the log file
<code>test_single_door.py</code>	I1	Only <code>executor.py</code> opens a DB connection	Any other module importing <code>sqlite3.connect</code> fails the test
<code>test_e2e.py</code>	Full pipeline	Piotr gets headcount and salary	Anna gets <code>[UNAVAILABLE]</code> for the CEO; blocked <code>send_email</code> reported

Telemetry check. `python -m tests.bench` sends a fixed prompt set and reports median and p95 latency per pipeline step, for the slides and the performance criterion.

Judge-friendly output. Test names read as sentences (`test_intern_cannot_read_ceo_salary`). The README shows the one command and an expected summary line.

15. Delivery plan

We plan for the earlier possible deadline and work in three tiers, timed as hours after the official start (H0). Feature freeze is three hours before the deadline.

Deadline. The English competition terms say work starts no earlier than 11:00 PM on 3 October and submissions close at 11:00 PM on 4 October. Another source reports 11:00 AM. We treat the earlier time as binding and confirm with the organizers on Discord at H0. The 50% phase-1 threshold means breadth across all five criteria beats depth in one.

Hour 1: model test

Run 10 fixed prompts against `llama3.2` and `qwen2.5:3b` through Ollama's OpenAI-compatible endpoint with the `query_data` tool. Measure for each model: share of data questions that call `query_data`, share of SQL that passes the validator, share of answers that use the placeholder, and median latency. The best model becomes `models.answer`.

- ☐ Hour-1 model test, result recorded in README
- ☐ Repo skeleton, stub model, `pytest` running green on an empty suite
- ☐ Deadline confirmed with organizers

Tier 1: MVP (H1 to H10)

- ☐ Auth and policy loader with reload, version hash and validation
- ☐ Masker (secrets, PII) and injection phrase list
- ☐ Signature feed matching on input, tool results and model output
- ☐ Proxy pass-through for plain chat
- ☐ Client tool authorization with argument rules
- ☐ `query_data` tool loop on the stub model
- ☐ SQL validator, authorizer (tables, columns, scope `self` and `all`), read-only executor with `set_authorizer`
- ☐ Placeholder-only disclosure and single-pass fill
- ☐ Deterministic output filter (secrets, PII, unbound placeholders)
- ☐ Basic budget and audit log
- ☐ Tests for every item above

Tier 2: complete (H10 to H17)

- ☐ Real Ollama end to end
- ☐ Judge model with hardened prompt and `on_failure`
- ☐ History re-masking and issued-value cache
- ☐ Protected-value index in the output filter

- ☐ Dashboard with the panels in section 12, auto-refresh, CSV export
- ☐ Model allowlist with digest pinning; model file scanner
- ☐ Disclosure with labels for `allow` users; scope `department`
- ☐ Profiles and `GET /policy/effective`

Tier 3: stretch

- ☐ Benchmark script and latency slide
- ☐ Scope enforcement by rewrite
- ☐ Separate aggregate permissions
- ☐ `fetch_feed` command
- ☐ Buffered SSE for `stream: true`
- ☐ Hash-chained audit log
- ☐ `all_denied_message`

Freeze (deadline minus 3 hours)

- ☐ README with one-command start and test command
- ☐ Demo script for the four worked examples in section 4
- ☐ Full test run, output captured for the slides
- ☐ Recorded demo video as a backup
- ☐ Slides, at most 10, exported to PDF
- ☐ HackTribe submission: title, team name, members (1 to 6), description, slides PDF, repository link

Slide outline (10 slides)

1. Problem and one-line answer
2. Architecture and trust boundary
3. One pipeline in ten steps
4. Deferred binding: Anna's example
5. Governed agency: blocked `send_email`
6. Policy and strictness profiles, live reload
7. Dashboard and audit
8. Test suite and OWASP coverage

9. Performance telemetry

10. Integration in one line and scalability path